

HTMAA Exam: Chess Robot

Jesper Terkildsen
IT University of Copenhagen
Copenhagen, Denmark
Email: jete@itu.dk

Allan N. Hadzimahovic
IT University of Copenhagen
Copenhagen, Denmark
Email: allh@itu.dk

Maria-Lavinia Otelea
IT University of Copenhagen
Copenhagen, Denmark
Email: mote@itu.dk

I. INTRODUCTION

The three of us have been fascinated by the evolution of chess engines, from IBM's early programs and Deep Blue to modern engines like Stockfish, even though none of us are skilled players. We were driven by this desire to explore whether we could build our own chess robot and potentially play against it. Making the robot accessible was just as important to us. Our goal was to make it possible for those who have physical limitations to play chess on an actual board without having to use a virtual interface. To support this, we included a basic keypad that allows players to easily enter their moves, making the game more inclusive while retaining the tactile and social characteristics of over the board chess.

II. BACKGROUND

We were fascinated by the 3D-printers at ITU because of their precision, versatility and simple design. Having actual 3D-printers also let us study stepper motors, mountings and assemblies very closely, which proved helpful. After studying the 3D-printers we went to projecthub.arduino.com to get more inspiration. We found multiple chess robots, but all of them had a robotic arm to move the chess pieces. Therefore, the novelty in our project is to incorporate mechanisms from a 3D-printer (belts, bearings etc.) into a chess robot. At projecthub.arduino.com, we found a similar concept named 'Botgammon Battle' [1] a robot that utilizes belts, stepper motors and an electromagnet to play backgammon. This robot helped greatly to conceptualize and scope our project.

III. REQUIREMENTS

The ILOs and guidelines made by the teachers set out some formal requirements for the robot regarding mechanisms, electronics, and embedded programming. We translated these into a concrete set of system requirements for our chess robot. It should use two stepper motors for precise horizontal movement on the XY plane and one servo motor for vertical movement in the Z-axis. The robot should use a hall effect sensor to detect the magnets attached to the top of the chess pieces. The robot should be able to lift pieces and move them over/above other pieces. The robot should be able to remove pieces if they are 'eliminated'. Finally, the robot should avoid executing illegal actions for example attempting to place two pieces on the same square, or moving a piece from an empty square.

An early design discussion was the level of "intelligence"

the robot should include. One option was to create a fully interactive opponent by integrating Stockfish and using a camera based computer vision system to read the player's moves directly from the board. This would make the robot feel autonomous, but it would also introduce too much complexity. The alternative was to focus on the robotics problem itself physical execution of moves while leaving decision making to the players. In this approach, two human players enter moves through a simple interface like the keypad and display, and the robot acts as the physical executor. Given the scope of the course and the timeline, we went with the second option. It allowed us to validate the complete pipeline (human input to motion planning to actuation to pickup/release).

IV. DESCRIPTION OF DEVICE

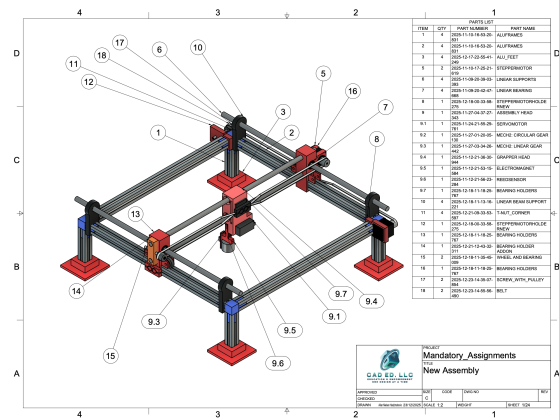


Fig. 1: Technical drawing of the Chess Robot.

A. Mechanics

The robot has three degrees of freedom which are split between the two mechanisms for 1) Horizontal movement and 2) Vertical movement. For the first mechanism 1), the two degrees of freedom come from movement along the x- and y-axis. Here, two stepper motors use a simple setup with belts, bearings, and pulleys to move the pieces around precisely. One stepper motor is fixed to the aluminum frame and produces the x-axis movement. The other stepper motor moves along with this x-axis movement while producing the

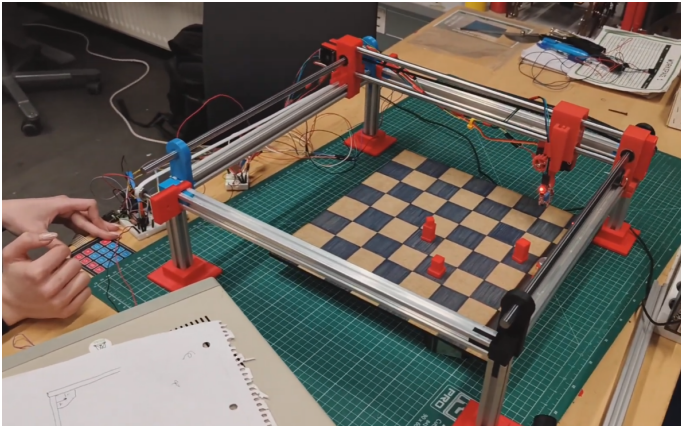


Fig. 2: Image of the robot from our demo video.

y-movement. For this mechanism, we utilized 3D-printed parts to a) combine the 4 horizontal and 4 vertical aluminum V-Slots (11), b) to hold the bearings (10, 13, 14, 16) c) fixate the stepper motors (8, 12, 16) and d) as feet for the vertical v-slots (3). The end of the belts are fixed to component (13) and (9.7) by the force of friction.

The second mechanism 2) controls the vertical movement of our assembly head (9), hence creating the third degree of freedom. We considered different linear actuators and chose a rack and pinion design. A gear (the pinion) is attached to the servo motor which engages the linear gear (the rack). Rotational movement from the servo motor is hereby converted into linear movement. We chose a servo motor because it can easily move up and down and because it holds its position when not powered/running. At the very end of the linear gear, an electromagnet and a Hall-effect sensor are attached.

B. Electronics

1) *Sensors:* We use a Hall-effect sensor to detect the magnetic fields of the magnets placed on top of the chess-pieces. When detecting a magnetic field above a threshold, the sensor sends a signal to the Arduino which then turns on the electromagnet to attract the chess piece. The sensor is mounted on the end of the Linear Gear (9.3) next to the electromagnet so that it follows the vertical movement of the Linear Gear. When the Assembly Head (9) has moved the chess piece the desired number of steps, the Arduino will turn off the electromagnet hence releasing the chess piece. Our Hall-effect sensor has 4 pins, VCC (5V), Ground, Analog output and Digital output. The sensor's analog output is proportional to the magnetic field it senses, hence increasing when it gets closer to the magnets of the chess pieces. Instead of a hall-effect sensor, we considered infrared and ultrasonic sensors to detect when the electromagnet was close enough to the chess pieces. However, an ultrasonic sensor has a wider 'beam' of detection which might falsely detect adjacent pieces. An infrared (IR) sensor is sensitive to light

and dependent on the surface of the chess pieces. Lastly an IR sensor alongside an IR receiver opposite it to generate a beam that, if broken, a piece must be present.

2) *Motor Driver:* We used two A4988 Stepper Drivers to control and power the two stepper motors. Firstly, a Stepper Driver can deliver enough current to the stepper motors. Secondly, it supports microsteps to control the precise movement of the chess pieces.

3) *Microcontroller:* We used an Arduino Uno to control all the electronics. It reads signals from the Hall-effect sensor and the keypad inputs to control the motors and electromagnet accordingly. It provides power for the sensor and servomotor while it is powered through a 5V power supply. The stepper motors and electromagnet require 12V power to function, so they are powered through external dedicated 12V power supply.

C. Software

We wrote the software in C++ and used the following libraries such as AccelStepper. It is an Arduino library designed for controlling stepper motors with support for acceleration, deceleration, and non-blocking operation. We have used it to provide smooth and coordinated control of the X and Y stepper motors. It reduces mechanical stress of the motors and avoids skipping steps by gradually increasing their speed using `setMaxSpeed()` and `setAcceleration()`. Additionally, the library makes it possible to translate chessboard row and column inputs straight into step locations by enabling absolute positioning using `moveTo()`. Additionally, it is important to continuously use `run()` for each motor inside the main loop to enable simultaneous movement of both motors while maintaining keypad responsiveness. (See code in the Annexes)

D. Actuators and Motion Control

The motion of the chess pieces across the board is achieved using two stepper motors configured in an XY belt-driven mechanism, adapted from the kinematic principles commonly used in 3D printers. Each stepper motor controls one axis of motion, X and Y respectively. This design enables a repeatable movement across a two-dimensional plane, which is essential for reliably picking up and placing chess pieces during gameplay.

Stepper motors were chosen due to their ability to move in micro-steps, providing high positional accuracy without requiring complex feedback systems such as encoders. In this implementation, the motors are driven through dedicated stepper motor drivers connected to an Arduino microcontroller. A microstepping driver such as the A4988 allows higher resolutions by allowing intermediate step locations.

The belt-driven XY configuration offers several advantages, including reduced mechanical complexity and low friction compared to lead-screw systems. By reusing a proven mechanical architecture from 3D printers, the system benefits from well-understood design principles. This approach also supports scalability and ease of maintenance, making it

well suited for a robotic chess player. Another advantage is represented by the fact that stepper motors also hold their torque, which allows the carriage (housing with the electromagnet) to maintain its position without mechanical locking mechanisms. Furthermore, stepper motors were chosen over servos due to their positional accuracy, predictable behavior, and widespread use in other belt-driven XY mechanisms. Additionally, they have no rotational limit as the servos have a max angle.

E. Motion Control Alternatives

Staying within the 3D printing world for our inspiration for motion, we did originally design our robot to use the CoreXY mechanism to move the carriage along the XY plane. This mechanism is commonly found in modern 3D printers who are not bed slingers. The central mechanism uses two stepper motors, as we are, but it moves both at the same time making it possible to do more dynamic and precise movement. Although the concept was mechanically interesting, attempting to implement it revealed a series of practical issues. Combined with an approaching deadline, we chose to discard it and proceed with our current implementation. For our purposes this was sufficient, since we did not require the more complex motion capabilities that a CoreXY setup could have provided.

A similar idea we discussed shortly at the beginning was going under the board and make the whole mechanism more hidden. This would have us only do XY movement with an electromagnet. We went away from this design partly due to the issue of how the springer piece would work in such a setup, as the rules allow it to jump over other pieces, and we also liked the idea of moving a piece away from the board.

A fundamentally different approach are the classical robotic arm for chess, while an elegant solution we wanted to explore something less popular.

V. RESULTS & ANALYSIS

Starting with the XY plane movement using stepper motors, while tested in isolation, not attached to the aluminum frame, we got the expected movement. After attaching it to the frame we noticed a significant problem with moving along the x-axis, the y-axis was fine. After testing different parts we determined that the problem stemmed from the aluminum frames build, specifically its high flexibility. This flexibility originated from the corner connections being used to bind the whole frame together not being very rigid, in assembly referenced as T-nut corners.

Initially, we used three 3D printed T-nut corners for each corner to connect the frames. When this failed, we ended up with our final design, as can be seen in the assembly, where we designed a 3D printed corner piece with a printed T-nut in all aluminum frame entries and a solid connecting block 3. This redesign proved better than the initial, but we didn't

gain the expected rigidity from it and still had issues with x-axis movement not being as consistent or smooth as we had expected from our design.

Regarding the new design, the 3-way geometry introduced a notable 3D-printing challenge namely layer line orientation. Our first attempt was to print the part flat on a face to minimize the amount of required support material. However, this made one of the arms extremely brittle, since the layer lines ran straight across its axis, creating a clear weakness along the layer boundaries. This led us to print the part in the orientation shown in Figure 3, where the layer lines are not aligned across any single arm. As a result, no direction becomes significantly weaker, at the cost of increased print time and additional support material.

In terms of our central carriage, we experienced semi-consistent results when it came to our magnetism mechanism. Since our design used a hall-sensor to detect the magnet on the chess piece before it activated the electromagnet, the precise position of the magnet was crucial.

Below is a graph showing 15 movements of the piece without any human interference in the position of the pieces. 5

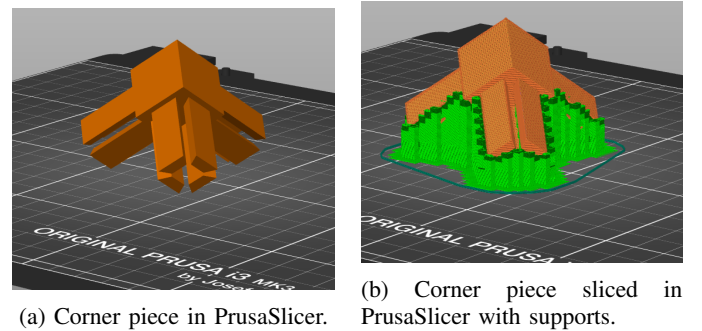


Fig. 3: Corner piece before and after slicing in PrusaSlicer.

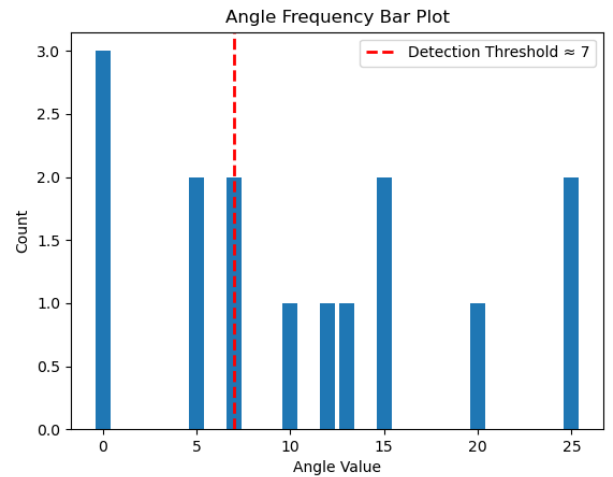


Fig. 4: Angle frequency experiment results.

The angles were obtained after each move, once the piece

had been released from the electromagnet. For each trial, we measured the piece's orientation relative to the hall sensor, treating the sensor direction as 0 degrees. We recorded only absolute angles (no negative values). We then noted whether the electromagnet could successfully pick up the piece again at that angle for the next movement. These measurements act as a proxy for how far the piece can be "misplaced" before the sensor no longer detects it.

Two main observations stand out from this small set of data points. First, the measured angles show substantial variation, spanning roughly 0 to 25 degrees. While the trials at 20 and 25 represent extreme cases, the overall spread still suggests that the current setup places pieces inconsistently.

Second, the sensor only appeared to detect pieces when they were within about 7 degrees of being orthogonal to the sensor. In other words, the sensing is quite sensitive to orientation, which makes the rest of the system activation equally unforgiving. Various fixes could be done for this, which will be mentioned in discussion.

In figure 5 an illustration of a chess piece and the different angles which might not activate the hall sensor.

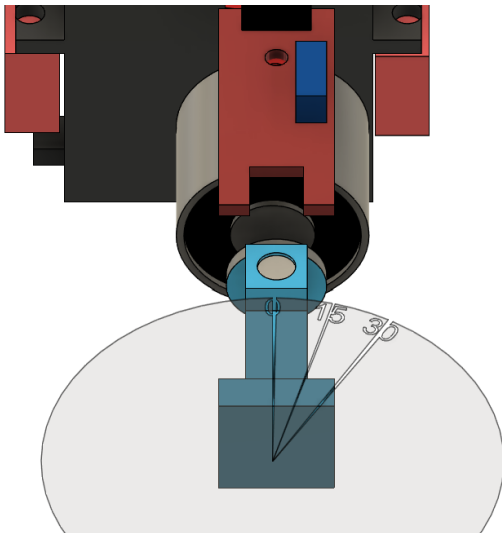


Fig. 5: Angle frequency experiment illustration in Fusion 360.

VI. DISCUSSION

We produced a working prototype of the chess machine where the core subsystems are largely functional most notably XY movement and reliable piece pickup. The next step we discussed internally, given more time, was to implement a full move sequence. Accept a start square and a target square, check whether the target square is occupied, and if so remove that piece to a "captured" area before moving the piece from the start square to the target square. This would bring us a system capable of executing a full move with minimal manual input from the users with automatic captures. Another scrapped idea we had was an LCD screen outputting to the player what they had inputted for a better and more transparent user experience.

As mentioned in the introduction one of the motivations for this project was to enable physically handicapped people to enjoy a more tactile board-game experience with a physical board. A future extension in addition to the keypad could be to interface their wheelchairs if they are in such a situation or in general make controlling our system easier. This alongside an extension with Stockfish AI could really make this feel like a more fulfilled and realized product for potential production.

Earlier we mentioned a major issue we encountered during development of this prototype was the flexibility of the aluminum frame due to the T-nut corners developed. We had two iterations for this piece both 3D printed in PLA. The major difficulty in this design was the 3-way structure, normal brackets from the lab only supported 2-way or in a normal corner. We did try out a 3-way corner bracket connector from the lab made from aluminum just as the frame [2], unfortunately all of those broke during development of the prototype when we moved it around. Which is why we ended up using a 3D printed solution even though it was clearly not perfect for this application.

In a final product, depending on how portable it needed to be, we would probably go with welding the frames together, or continue figuring out alternatives for better corner support.

As mentioned earlier, a different overall approach such as placing the motion system beneath the board could also reduce many of these mechanical challenges. An under board design would likely be both more elegant and more seamless than our relatively bulky prototype, and better for a mass production setting.

In the central carriage (electromagnet, hall sensor, and servo), the main issue was frequent misplacement of the pieces after a move. This largely comes down to the limited region where the hall sensor can reliably detect the piece, combined with how the electromagnet contacts the piece during pickup and release. With more adjustment and fine tuning especially of the hall sensor's sensitivity and positioning we likely could improve the results noticeably. A more severe issue here is the design in on itself. We originally wanted to take this approach to make it possible for us to have multiple height pieces, as in a traditional game of chess, and have it pick up and move them across the board.

First issue is the height we can gain, the current servo has a relatively limited range it can rotate. Thus a smarter mechanism from the rotation or a more freely moving motor like stepper might be appropriate.

When it comes to detecting the pieces height using the hall sensor and magnet, one way to avoid the "small detection area" problem would be to give each piece a magnetic field that surrounds it (detectable from any orientation). However, that approach would also complicate both the design and the visual appearance of the pieces. A related question is why we did not place a permanent magnet on top of each piece instead of using only a ferromagnetic insert. This was, in fact, our initial approach. In practice, we found

that pieces with a permanent magnet were difficult for the electromagnet to release consistently. The most likely cause was residual magnetism in the electromagnet, meaning it remained slightly magnetized even when powered off. This behavior is a common issue in electromagnet setups and can lead to unintended “sticking” during release. [3].

The final prototype is a robot capable of executing a subset of chess moves. With some movement challenges at times. It uses two stepper motors for movement in the XY plane and a servo motor for vertical motion along the Z-axis. Piece pickup is handled using a Hall effect sensor for detection and an electromagnet for gripping.

After adjusting the piece heights, the servo’s travel is just sufficient to lift a piece above the others, allowing it to move across the board without frequent collisions. The robot does not yet remove captured pieces, but the required foundation XY movement and pickup/release is in place, which also allows for the player versus player feature. Similarly, the system currently has no built in rule checking, meaning it does not validate whether a move is legal or whether inputs are invalid.

VII. PROTOTYPE YOUTUBE LINK

<https://www.youtube.com/watch?v=vQstGV-e8uo>

REFERENCES

- [1] alonborn, “Botgammon battle,” <https://projecthub.arduino.cc/alonborn/botgammon-battle-f59e0d>.
- [2] PZRT, “4-pack 3030 series 3-way end corner bracket connector with screws for standard 8 mm t-slot aluminum extrusion profile,” Online product listing, Amazon, <https://www.amazon.com/PZRT-Connector-Standard-Aluminum-Extrusion/dp/B07C8G4NXL>.
- [3] K. Magdy, “Arduino electromagnet control,” <https://deepbluembedded.com/arduino-electromagnet-control-circuit-code-example/>.

VIII. ANNEXES

A. Bill of materials (BOM)

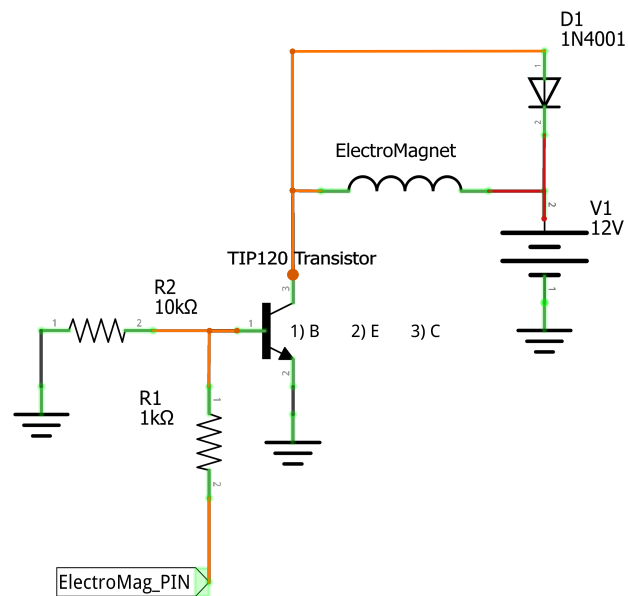
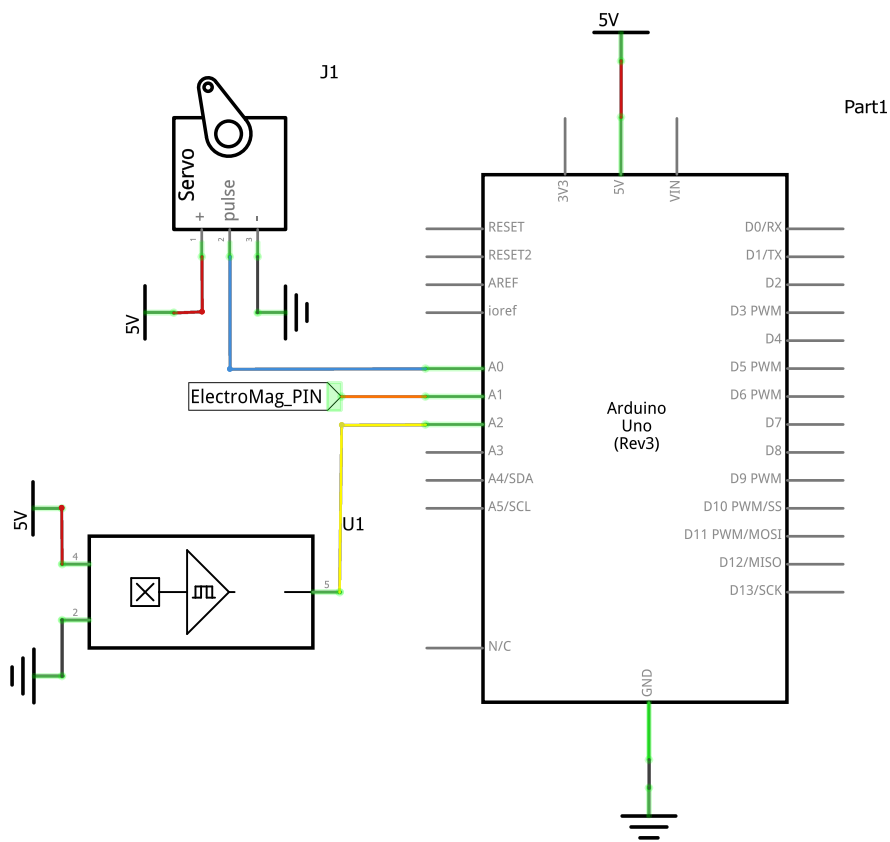
TABLE I: BOM Electronics

Amount	Component	Price Total	Link
1	Corona DS558HV Servo Motor	81.41 DKK	https://www.aliexpress.com/...
2	Nema 11 Stepper Motor	189.00 DKK	https://www.elektrotronik.com/...
1	Arduino Uno	65.00 DKK	https://ardustore.dk/
1	12V Power Supply	75.00 DKK	https://ardustore.dk/
1	5V Power Supply	54.00 DKK	https://ardustore.dk/
2	A4988 Stepper Driver	20.00 DKK	https://ardustore.dk/
1	Hall effect sensor	54.00 DKK	https://ardustore.dk/
1	Electromagnet	51.25 DKK	https://arduinotech.com/...
1	Matrix Keypad 4x4 Membran Switch	20.00 DKK	https://ardustore.dk/
1	Breadboard	45.00 DKK	https://ardustore.dk/

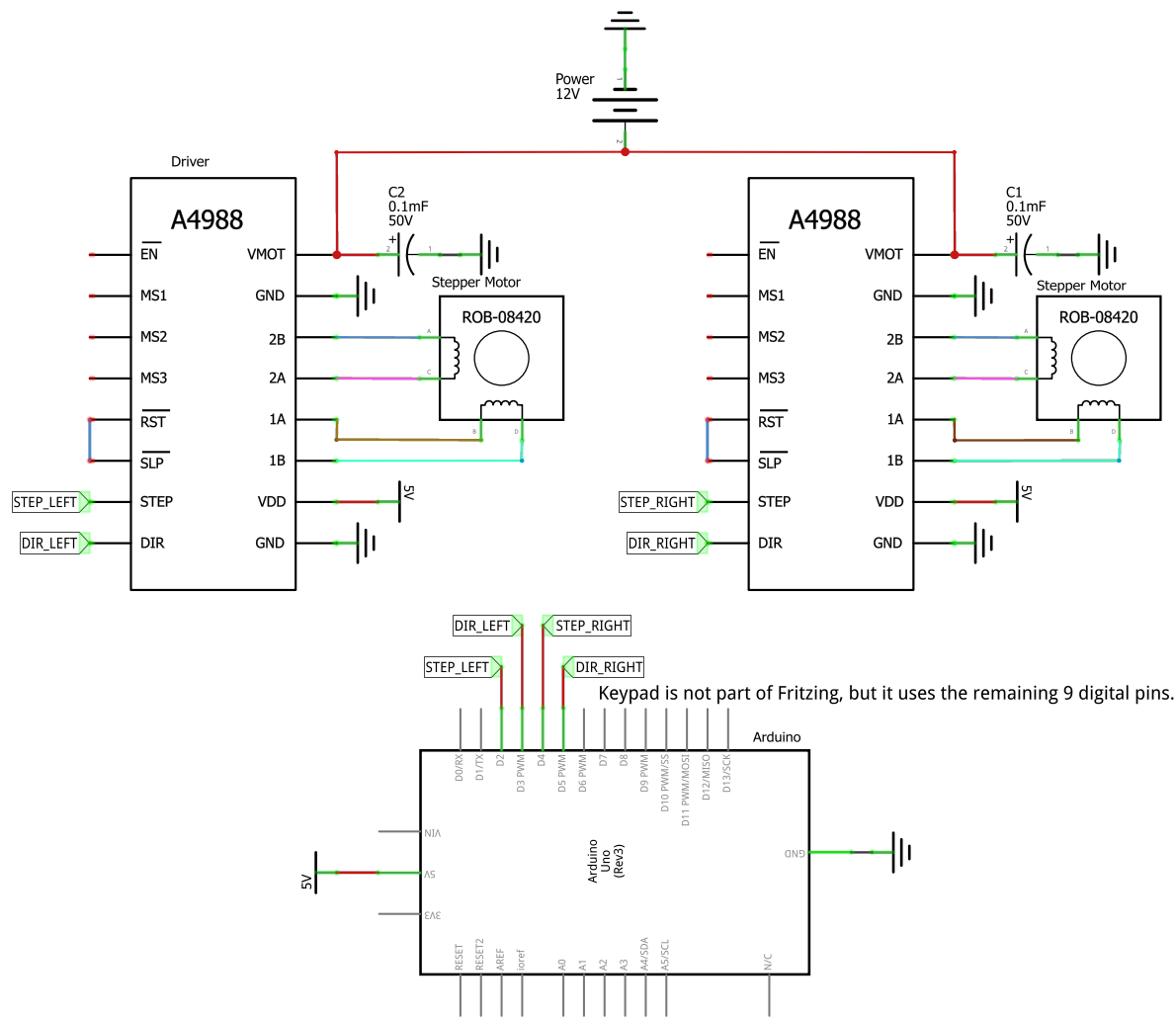
TABLE II: BOM Other

Amount	Component	Price Total	Link
1	1KG PLA	89.00 DKK	https://3dekspten.dk/products/sunlu-1kg-pla
300 mm	Aluminium Profile	70.75 DKK	https://www.amazon.com.be/-/en/Aluminium-Profile-300mm-30mm-x-30mm-3030-Profile-4-Pack-3030-Series-3-Way-End-Corner-Bracket-Connector-With-Screws-For-Standard-8-Mm-T-Slot-Aluminum-Extrusion-Profile/dp/B07C8G4NXL
250 mm	Linear beam	59.75 DKK	https://makersupplies.dk/da/maskinkomponenter/lineaer-beam-250mm-30mm-x-30mm-3030-Profile-4-Pack-3030-Series-3-Way-End-Corner-Bracket-Connector-With-Screws-For-Standard-8-Mm-T-Slot-Aluminum-Extrusion-Profile
4	Linear Slide Bearing	149.95 DKK	https://3dekspten.dk/products/myt-s-bearinger
2	Wheels	42.00 DKK	https://3do.dk/en/bearings/438-open-bearing
1	Belt	79.00DKK	https://3dekspten.dk/products/creality-belt
4	Pulley	220.00 DKK	https://3dekspten.dk/products/spide-pulley

B. Schematics



fritzing



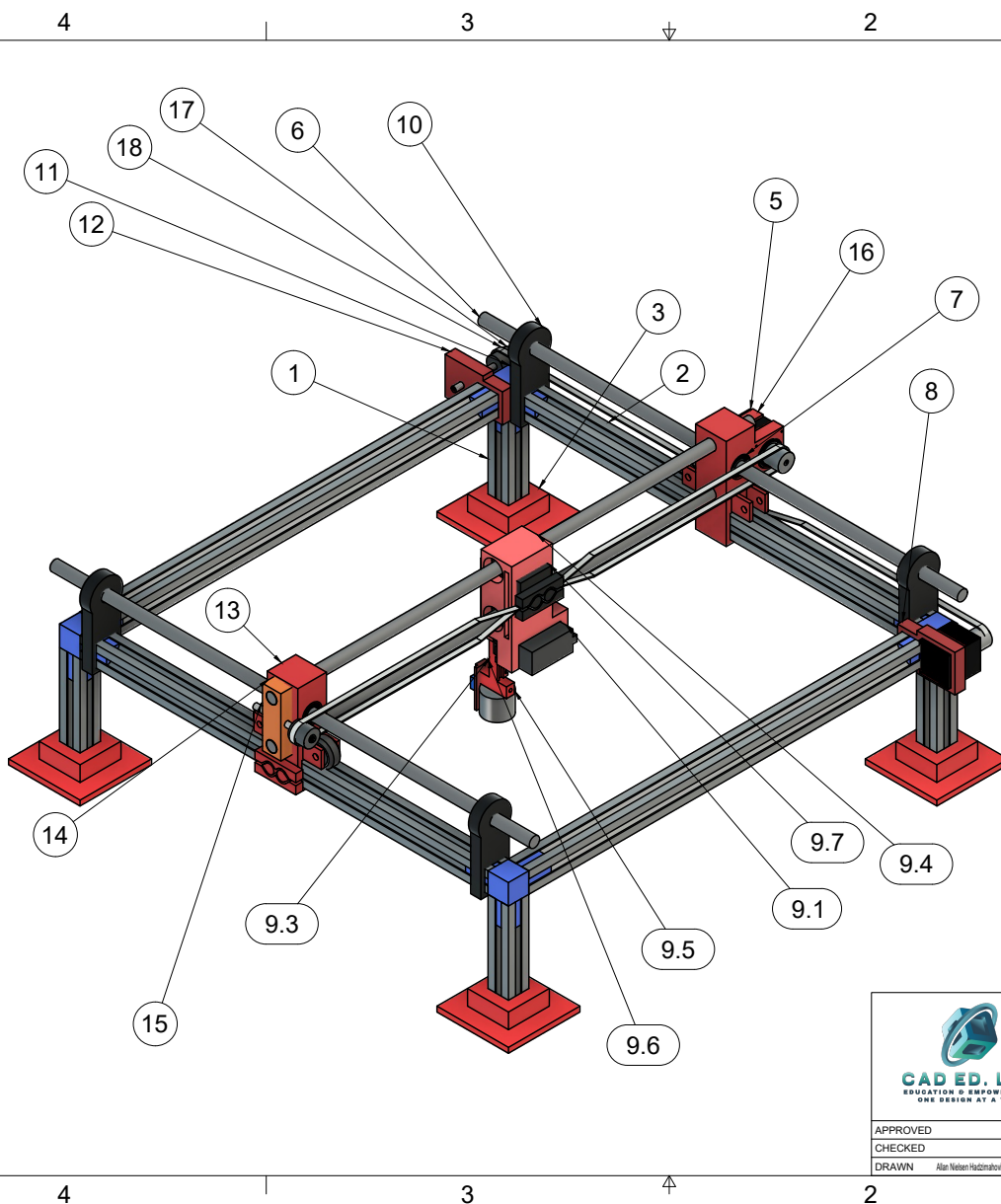
C. Technical Drawing

D

C

B

A



PARTS LIST			
ITEM	QTY	PART NUMBER	PART NAME
1	4	2025-11-10-16-53-20-831	ALUFRAMES
2	4	2025-11-10-16-53-20-831	ALUFRAMES
3	4	2025-12-17-22-55-41-249	ALU_FEET
5	2	2025-11-10-17-25-21-619	STEPPER MOTOR
6	4	2025-11-09-20-39-03-393	LINEAR SUPPORTS
7	4	2025-11-09-20-42-47-668	LINEAR BEARING
8	1	2025-12-18-00-33-58-275	STEPPER MOTOR HOLDER NEW
9	1	2025-11-27-04-37-27-343	ASSEMBLY HEAD
9.1	1	2025-11-24-21-55-29-761	SERVOMOTOR
9.2	1	2025-11-27-01-20-05-130	MECH2: CIRCULAR GEAR
9.3	1	2025-11-27-03-34-26-442	MECH2: LINEAR GEAR
9.4	1	2025-11-12-21-36-30-944	GRAPPER HEAD
9.5	1	2025-11-12-21-53-15-584	ELECTROMAGNET
9.6	1	2025-11-12-21-56-23-284	REED SENSOR
9.7	1	2025-12-18-11-18-25-767	BEARING HOLDERS
10	4	2025-12-18-11-13-16-221	LINEAR BEAM SUPPORT
11	4	2025-12-21-09-33-53-597	T-NUT_CORNER
12	1	2025-12-18-00-33-58-275	STEPPER MOTOR HOLDER NEW
13	1	2025-12-18-11-18-25-767	BEARING HOLDERS
14	1	2025-12-21-12-43-32-311	BEARING HOLDER ADDON
15	2	2025-12-18-11-35-45-009	WHEEL AND BEARING
16	1	2025-12-18-11-18-25-767	BEARING HOLDERS
17	2	2025-12-23-14-35-07-854	SCREW_WITH_PULLEY
18	2	2025-12-23-14-55-56-490	BELT



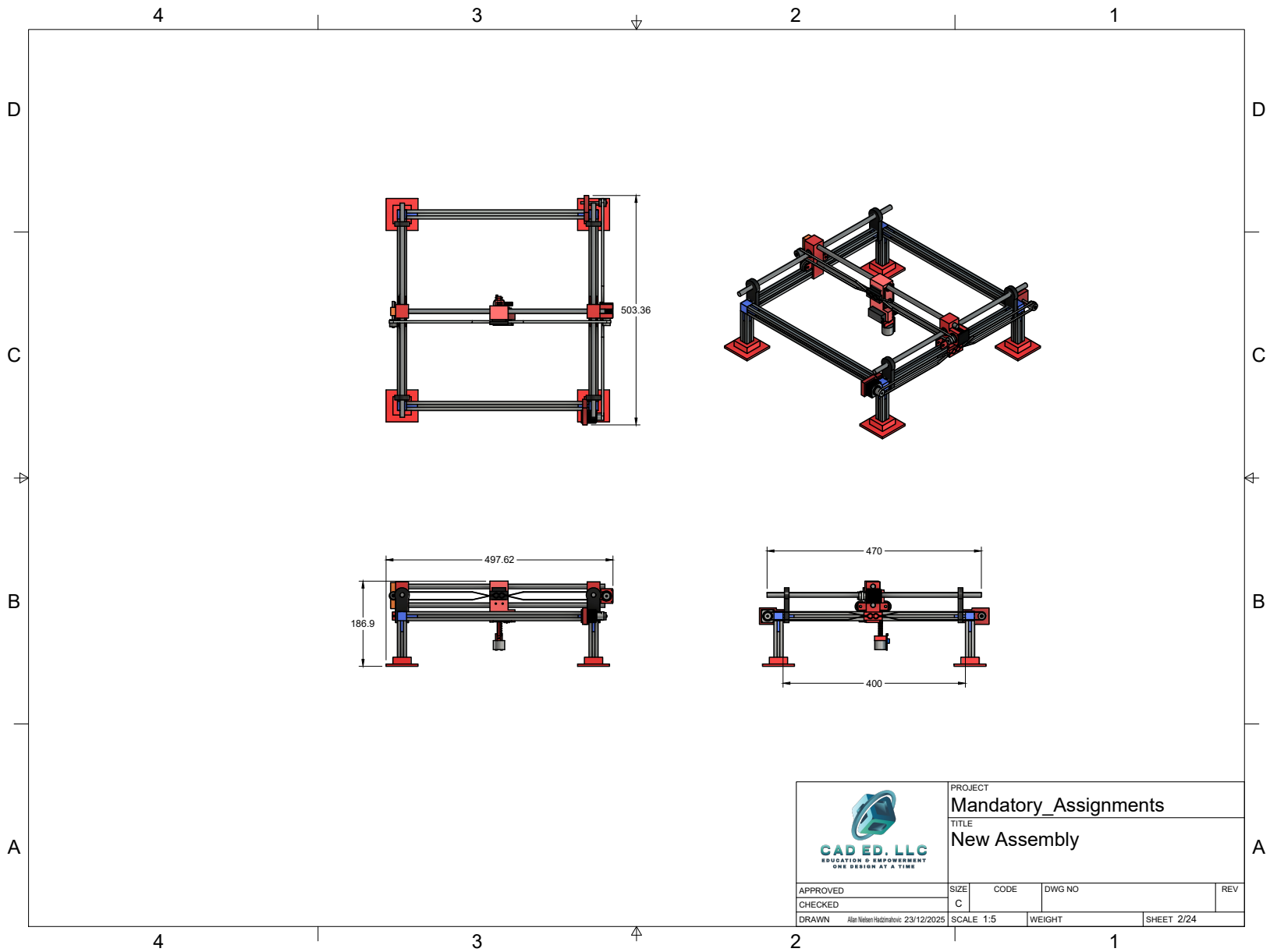
PROJECT		Mandatory_Assignments	
TITLE		New Assembly	
APPROVED	SIZE	CODE	DWG NO
CHECKED	C		
DRAWN	SCALE 1:2	WEIGHT	SHEET 1/24

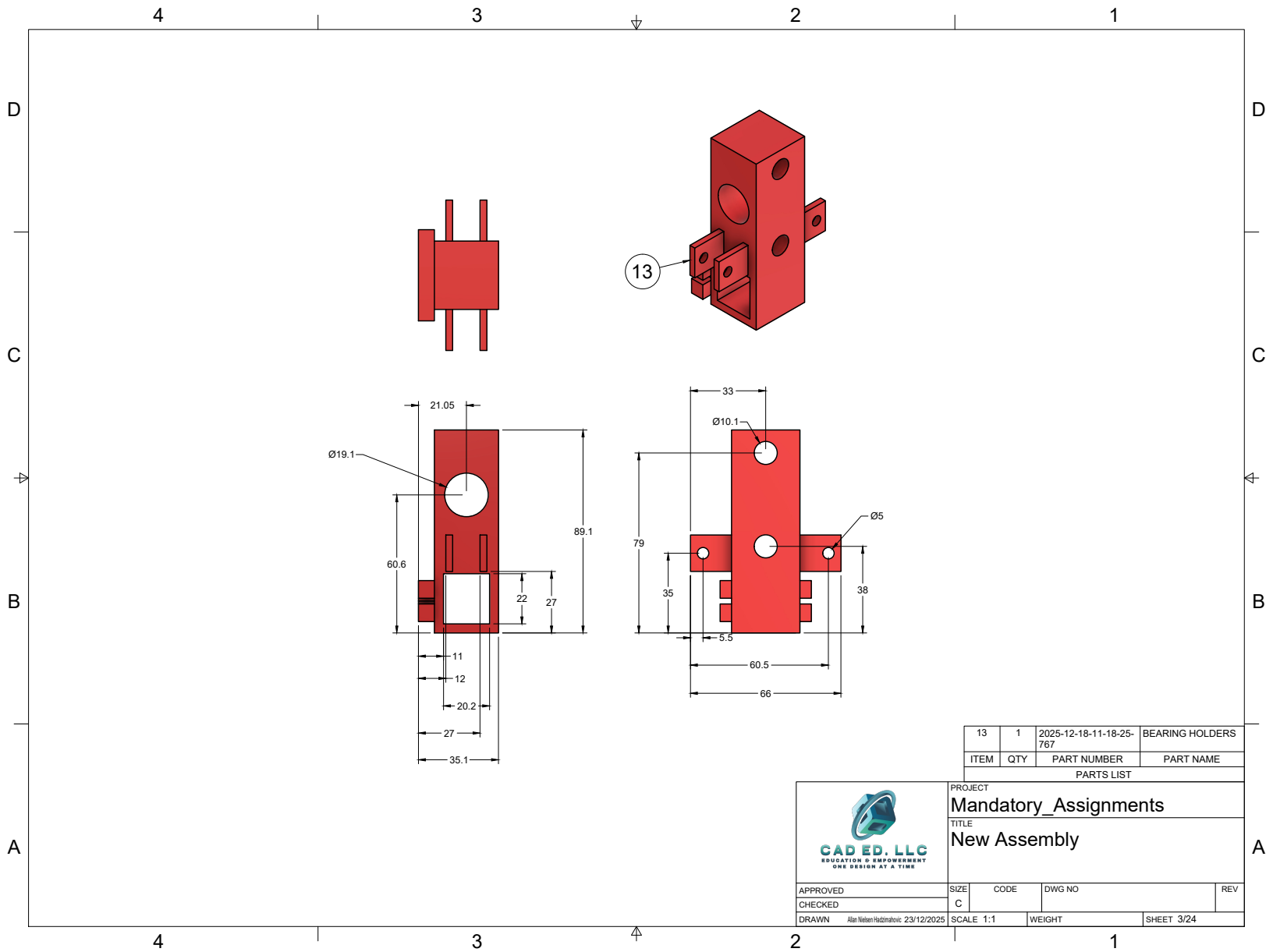
D

C

B

A

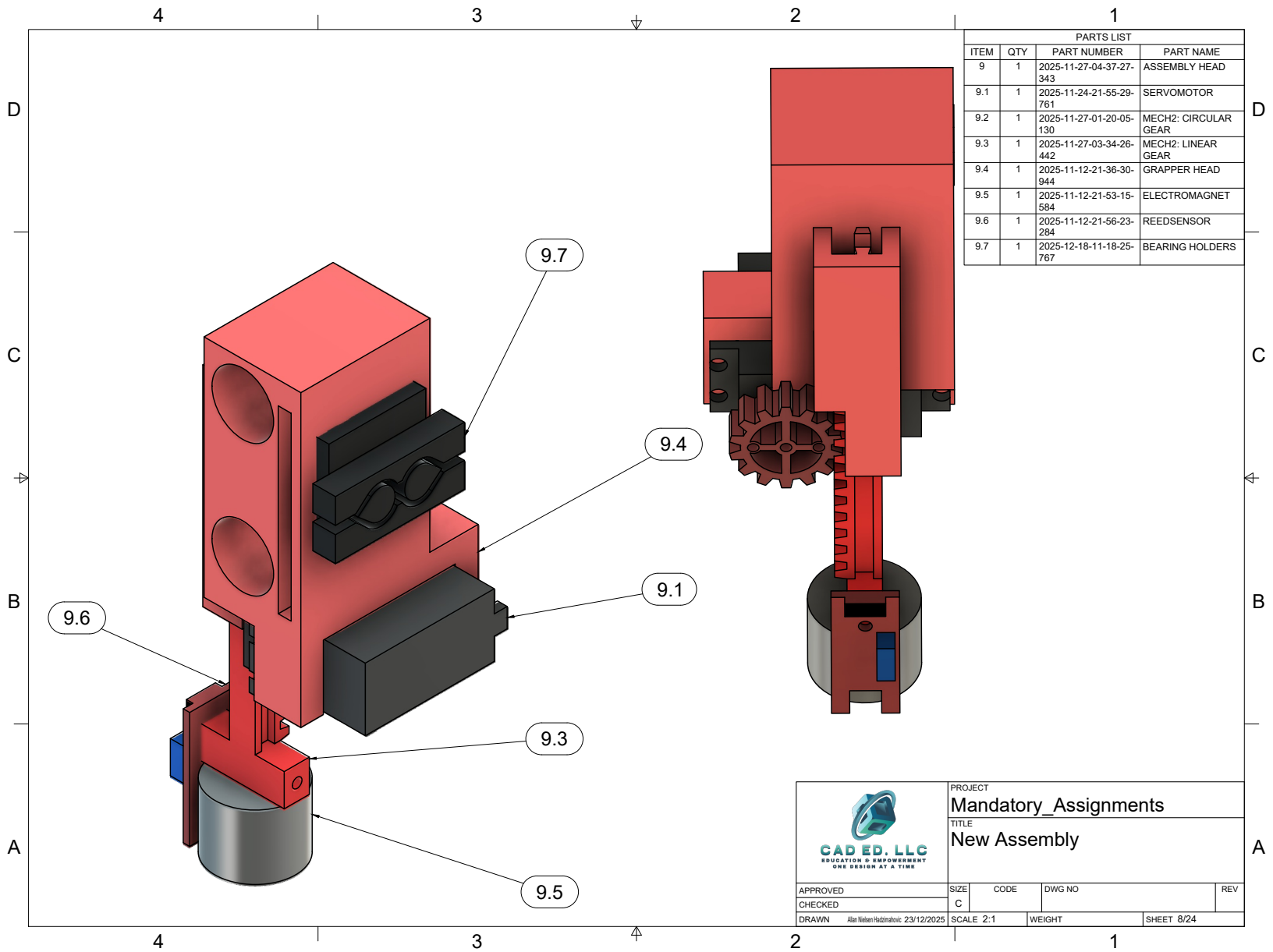




13	1	2025-12-18-11-18-25-767	BEARING HOLDERS
ITEM	QTY	PART NUMBER	PART NAME

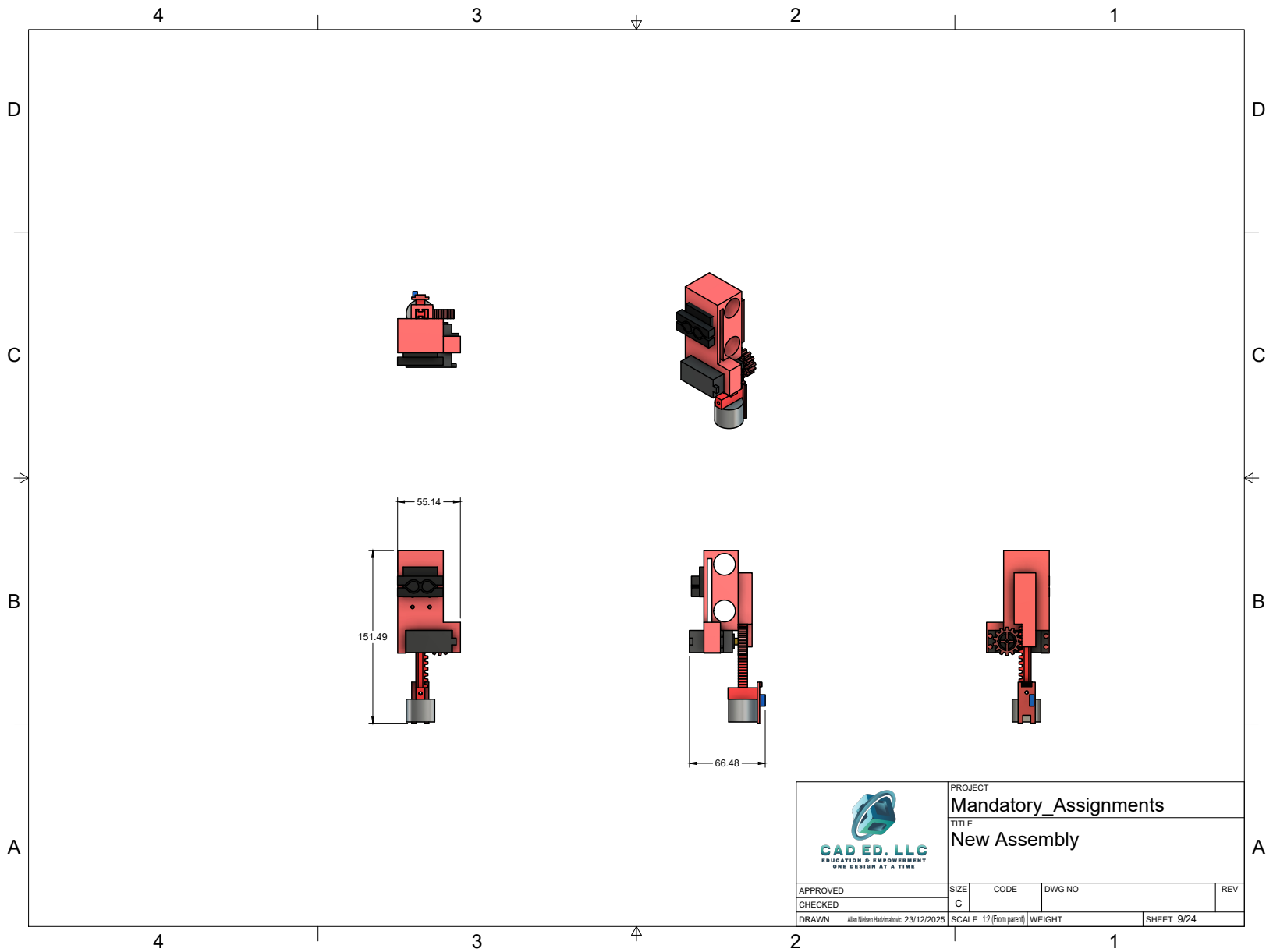
PARTS LIST

 CAD ED, LLC EDUCATION & EMPOWERMENT ONE DESIGN AT A TIME	PROJECT			
	Mandatory_Assignments			
	TITLE			
New Assembly				A
APPROVED	SIZE	CODE	DWG NO	
CHECKED	C			
DRAWN	Alan Nielsen	23/12/2025	SCALE 1:1	WEIGHT
			SHEET	3/24

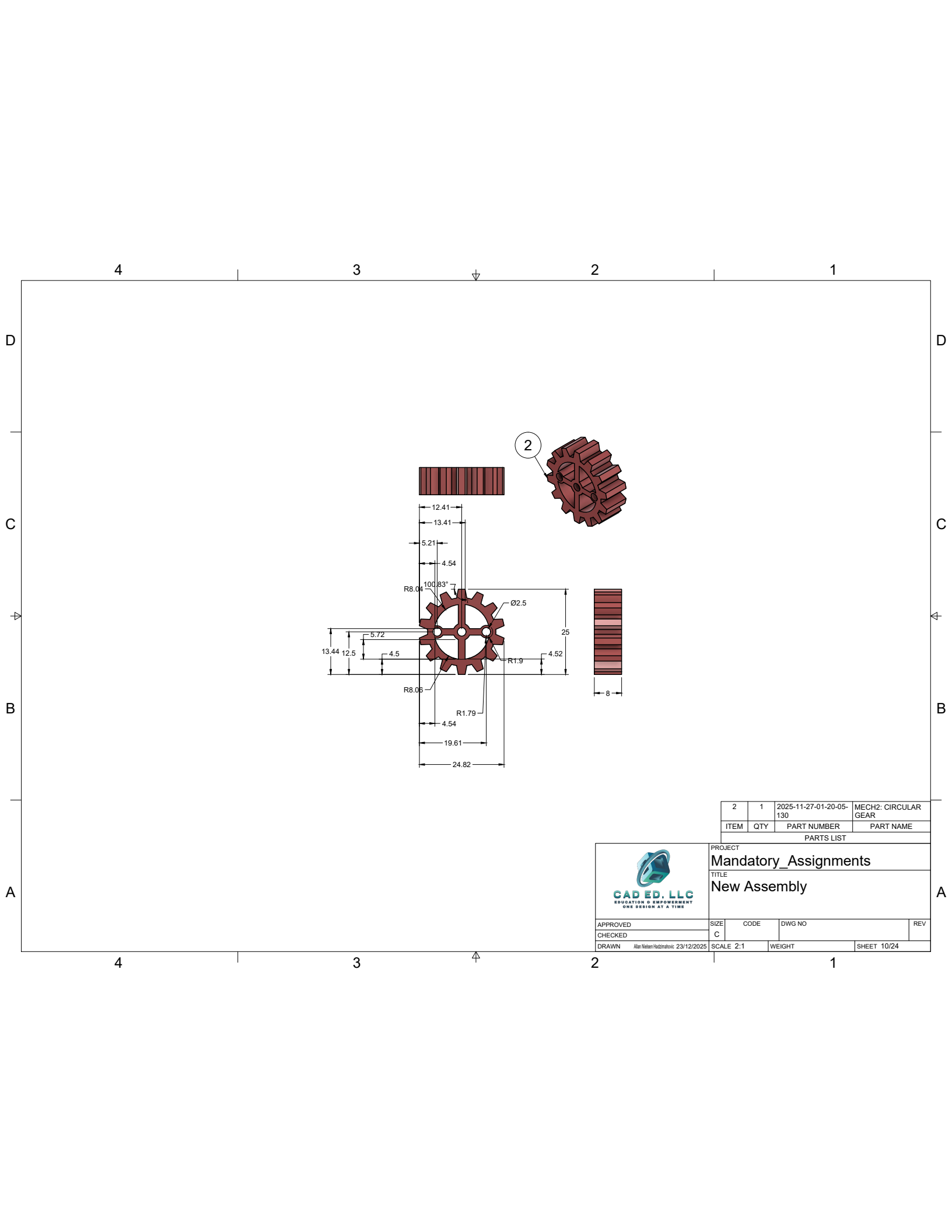


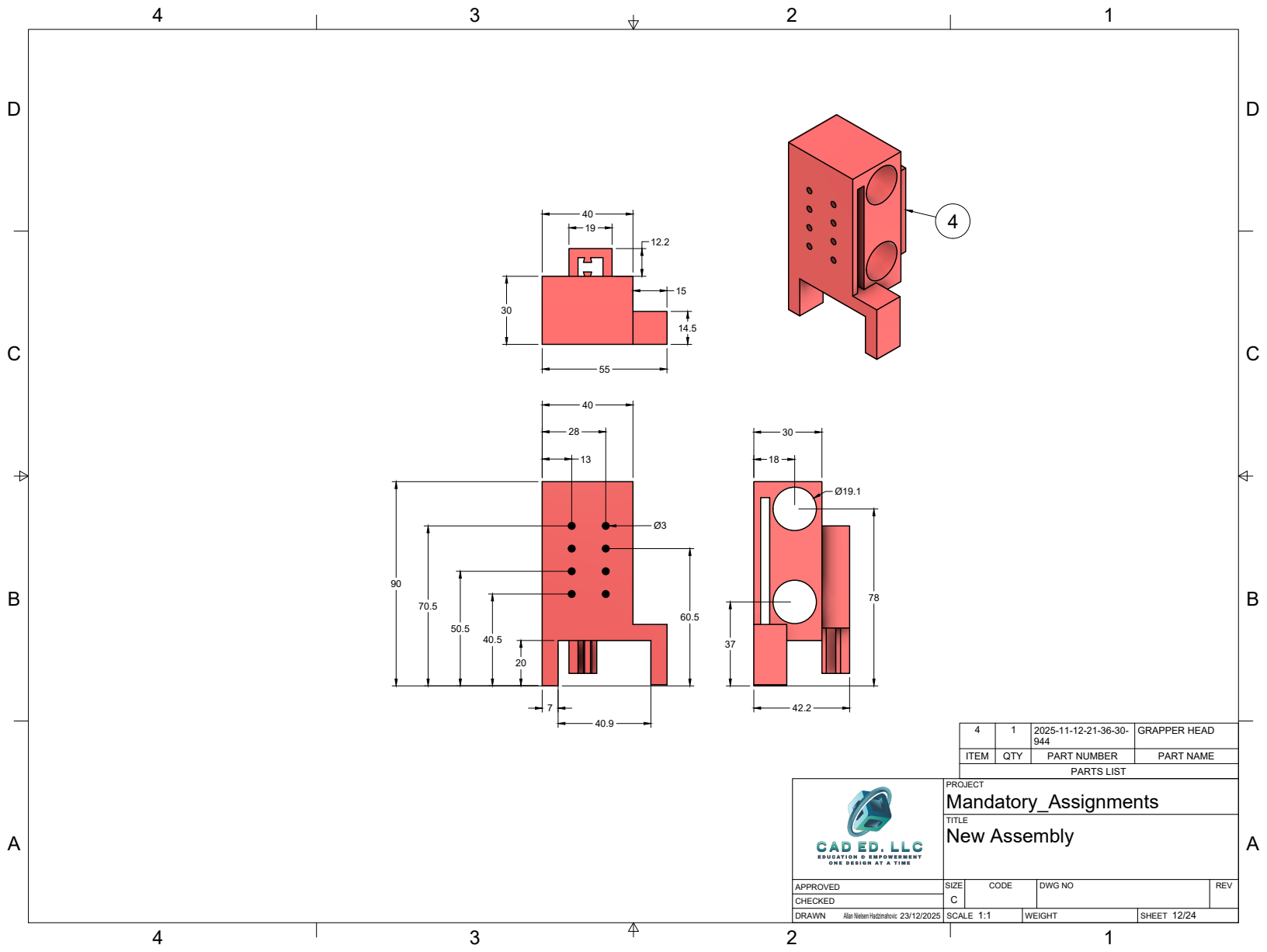
PARTS LIST			
ITEM	QTY	PART NUMBER	PART NAME
9	1	2025-11-27-04-37-27-343	ASSEMBLY HEAD
9.1	1	2025-11-24-21-55-29-761	SERVOMOTOR
9.2	1	2025-11-27-01-20-05-130	MECH2: CIRCULAR GEAR
9.3	1	2025-11-27-03-34-26-442	MECH2: LINEAR GEAR
9.4	1	2025-11-12-21-36-30-944	GRAPPER HEAD
9.5	1	2025-11-12-21-53-15-584	ELECTROMAGNET
9.6	1	2025-11-12-21-56-23-284	REEDSENSOR
9.7	1	2025-12-18-11-18-25-767	BEARING HOLDERS

 CAD ED, LLC EDUCATION & EMPOWERMENT ONE DESIGN AT A TIME	PROJECT		
	Mandatory_Assignments		
APPROVED CHECKED DRAWN	TITLE		
	New Assembly		
SCALE 2:1		CODE	REV
WEIGHT		DWG NO	SHEET 8/24



 CAD ED, LLC EDUCATION & EMPOWERMENT ONE DESIGN AT A TIME	PROJECT Mandatory_Assignments			
	TITLE New Assembly			
APPROVED	SIZE	CODE	DWG NO	REV
CHECKED	C			
DRAWN	Alan Nielsen-Pedersen 23/12/2025	SCALE 1:2 (From parent)	WEIGHT	SHEET 9/24

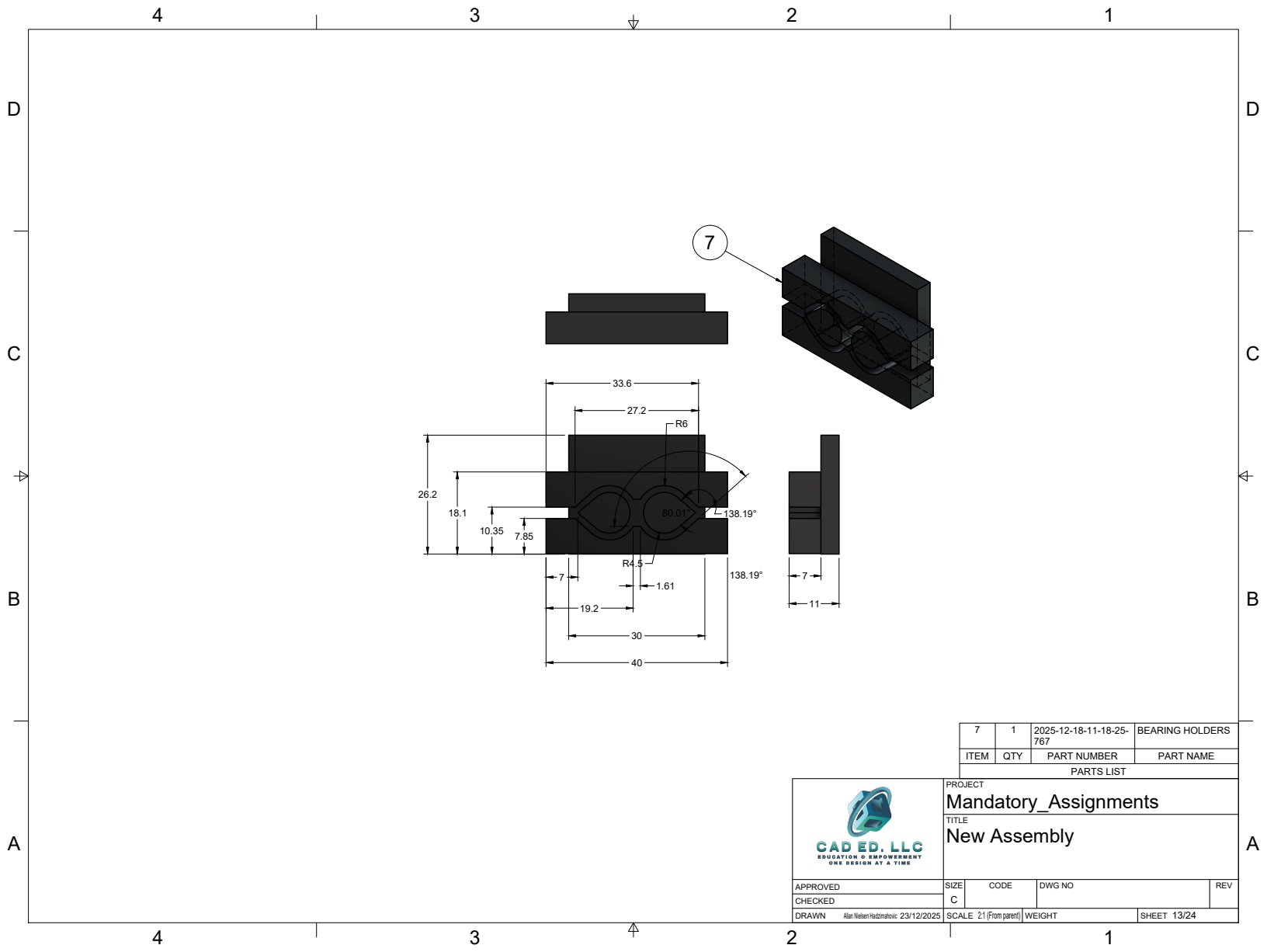


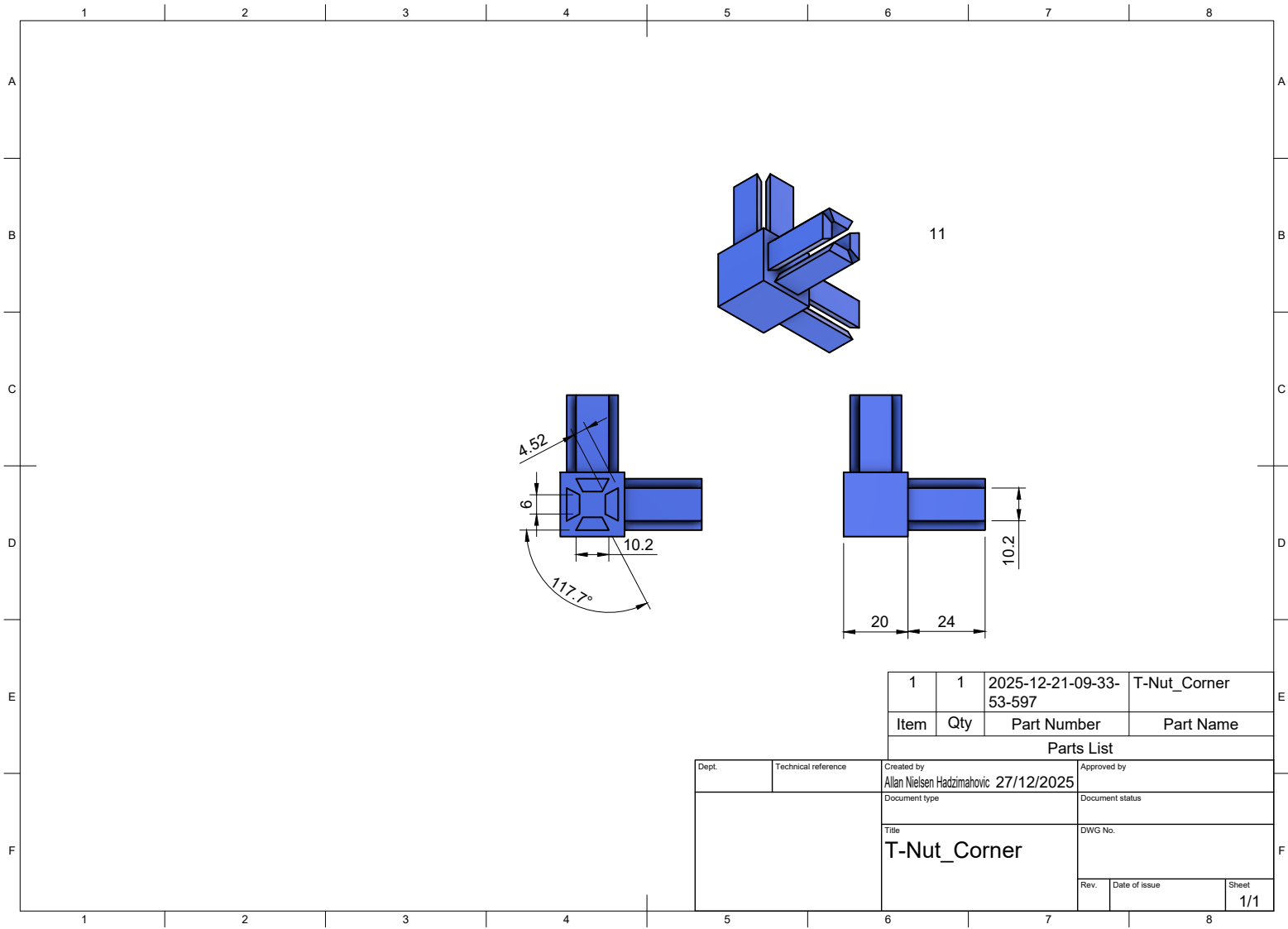


4	1	2025-11-12-21-36-30-944	GRAPPER HEAD
ITEM	QTY	PART NUMBER	PART NAME

PARTS LIST

 CAD ED, LLC EDUCATION & EMPOWERMENT ONE DESIGN AT A TIME	PROJECT			
	Mandatory_Assignments			
	TITLE			
New Assembly				REV
APPROVED	SIZE	CODE	DWG NO	
CHECKED	C			
DRAWN	Alan Nielsen	23/12/2025	SCALE 1:1	WEIGHT
			SHEET 12/24	





1	1	2025-12-21-09-33-53-597	T-Nut_Corner
Item	Qty	Part Number	Part Name

Parts List

Dept.	Technical reference	Created by Allan Nielsen Hadzimahovic 27/12/2025	Approved by		
		Document type	Document status		
		Title T-Nut_Corner	DWG No.		
			Rev.	Date of issue	Sheet 1/1

D. Code

The code file is also included in the additional files zip for easier reading.

```
Combined_Code.ino
1 #include <AccelStepper.h>
2 #include <Keypad.h>
3 #include <Servo.h>
4
5 // ----- Servo (2) and Electromagnet -----
6 Servo servo;
7 const uint8_t SERVO_PIN = A0;
8 const uint8_t ELECTROMAG_PIN = A1;
9 const uint8_t HALL_PIN = A2;
10
11 const int MIN_POS = 30;
12 const int MAX_POS = 120;
13 const int STEP = 1;
14 const unsigned long STEP_MS = 10;
15
16 const int HALL_ACTIVE_STATE = LOW;
17
18 enum State { LOWERING, RELEASING, LIFTING };
19 State state = LOWERING;
20
21 int pos = MAX_POS;
22
23 // In PICK mode avoid "instant pick" if hall is already active at the top.
24 // Start unarmed, then arm once hall go inactive during the descent.
25 bool armed = false;
26 unsigned long lastStepMs = 0;
27
28 bool magnetOn = false; // OFF->next actuation is PICK, ON->next actuation is PLACE
29 unsigned long releaseStartMs = 0;
30 const unsigned long RELEASE_DELAY_MS = 250;
31
32 const unsigned long PICK_TIMEOUT_MS = 6000; // max time allowed for a pick actuation
33 const unsigned long PLACE_TIMEOUT_MS = 6000; //
34 unsigned long actuationStartMs = 0;
35 // ----- Steppers (X and Y) -----
36 AccelStepper stepperY(AccelStepper::DRIVER, 5, 4); // STEP, DIR
37 AccelStepper stepperX(AccelStepper::DRIVER, 2, 3); // STEP, DIR
38
39 // CALIBRATE THESE
40 const long STEPS_PER_SQUARE_X = 220; // steps to move 1 column
41 const long STEPS_PER_SQUARE_Y = 220; // steps to move 1 row
42 // ----- Keypad -----
43 const byte ROWS = 4;
44 const byte COLS = 4;
45
46 char keys[ROWS][COLS] = {
47   {'1','2','3','A'},
48   {'4','5','6','B'},
49   {'7','8','9','C'},
50   {'*','0','*','D'}
51 };
52
53 Combined_Code.ino
54 // ----- Input state -----
55 char buf[3] = {0, 0, 0}; // two digits + null
56 byte idx = 0;
57
58 // Sequence control
59 bool moveRequested = false;
60 bool actuating = false;
61
62 void clearInput() {
63   idx = 0;
64   buf[0] = buf[1] = buf[2] = 0;
65   Serial.println("Cleared input");
66 }
67
68 bool hallActive() {
69   return digitalRead(HALL_PIN) == HALL_ACTIVE_STATE;
70 }
71
72 void setMagnet(bool on) {
73   digitalWrite(ELECTROMAG_PIN, on ? HIGH : LOW);
74   magnetOn = on;
75 }
76
77 void startActuation() {
78   // Reset computation state for this actuation
79   state = LOWERING;
80
81   // Start NOT armed | Arm once the hall is seen inactive during descent.
82   armed = false;
83
84   lastStepMs = 0;
85
86   // Always start from up
87   pos = MAX_POS;
88   servo.write(pos);
89
90   actuating = true;
91   actuationStartMs = millis();
92
93   if (!magnetOn) Serial.println("Arrived -> PICK (magnet will turn ON when hall triggers)");
94   else Serial.println("Arrived -> PLACE (magnet will turn OFF at bottom)");
95 }
96
97
98
99
100
```

```
Combined_Code.ino
98   else Serial.println("Arrived -> PLACE (magnet will turn OFF at bottom)");
99 }
100
101 void stopActuation(const __FlashStringHelper* reason) {
102   actuating = false;
103   moveRequested = false;
104   state = LOWERING;
105   Serial.println(reason);
106 }
107
108 void runActuation() {
109   unsigned long now = millis();
110
111   // Safety timeout so we never hang forever
112   if (!magnetOn) { // PICK mode
113     if (now - actuationStartMs > PICK_TIMEOUT_MS) {
114       stopActuation(F("Action timeout (PICK)"));
115       return;
116     }
117   } else { // PLACE mode
118     if (now - actuationStartMs > PLACE_TIMEOUT_MS) {
119       stopActuation(F("Action timeout (PLACE)"));
120       return;
121     }
122   }
123
124   switch (state) {
125     case LOWERING: {
126       // Step servo down
127       if (now - lastStepMs >= STEP_MS) {
128         lastStepMs = now;
129         pos -= STEP;
130         if (pos < MIN_POS) pos = MIN_POS;
131         servo.write(pos);
132       }
133
134       // ----- PICK mode (magnet currently OFF) -----
135       if (!magnetOn) {
136         bool hall = hallActive();
137
138         // Arm once hall is NOT active (this prevents instant trigger if hall is active at the top)
139         if (!hall) armed = true;
140
141         // When hall triggers (and we are armed), turn magnet ON and start lifting
142         if (hall && armed) {
143           armed = false;
144           setMagnet(true); // magnet ON and STAYS ON until next actuation (place)
145           state = LIFTING;
146           break;
147         }
148
149         // If we hit bottom and never got a hall trigger, abort pick and lift anyway (magnet stays OFF)
150         if (pos <= MIN_POS) {
151           Serial.println("Pick failed (no hall). Lifting without magnet.");
152           state = LIFTING;
153         }
154       }
155
156       // ----- PLACE mode (magnet currently ON) -----
157       else { // When fully lowered, turn magnet OFF to release
158         if (pos <= MIN_POS) {
159           setMagnet(false); // magnet OFF (release)
160           releaseStartMs = now;
161           state = RELEASING; // small pause before lifting
162         }
163         break;
164       }
165     }
166     case RELEASING: {
167       // Let the piece actually drop before lifting
168       if (now - releaseStartMs >= RELEASE_DELAY_MS) {
169         state = LIFTING;
170       }
171       break;
172     }
173     case LIFTING: {
174       // Step servo up
175       if (now - lastStepMs >= STEP_MS) {
176         lastStepMs = now;
177         pos += STEP;
178         if (pos > MAX_POS) pos = MAX_POS;
179         servo.write(pos);
180       }
181
182       // Finish actuation once back at the top
183       if (pos >= MAX_POS) {
184         actuating = false;
185         moveRequested = false;
186         state = LOWERING;
187         Serial.println("Actuation done");
188       }
189       break;
190     }
191   }
192 }
193
194 // ----- Setup -----
195 void setup() {
196   Serial.begin(9600);
197   Serial.println("Chess XY move: enter RC (e.g. 12), press # to go. * clears.");
198
199   stepperY.setMaxSpeed(1500);
200   stepperX.setAcceleration(800);
201
202   // ----- End of Setup -----
203 }
```

```

194 ////////////////////////////////////////////////// Setup //////////////////////////////////////
195 void setup() {
196   Serial.begin(9600);
197   Serial.println("Chess XY move: enter RC (e.g. 12), press # to go. * clears.");
198
199   stepperX.setMaxSpeed(1500);
200   stepperX.setAcceleration(800);
201
202   stepperY.setMaxSpeed(1500);
203   stepperY.setAcceleration(800);
204
205   // Start at home (row,col)
206   stepperX.setCurrentPosition(0);
207   stepperY.setCurrentPosition(0);
208
209   // Servo
210   servo.attach(SERVO_PIN);
211   servo.write(pos);
212
213   // Electromagnet
214   pinMode(ELECTROMAG_PIN, OUTPUT);
215   setMagnet(raise);
216
217   // Hall sensor
218   pinMode(HALL_PIN, INPUT_PULLUP);
219 }
220
221 void loop() {
222   // Always keep steppers running
223   stepperX.run();
224   stepperY.run();
225
226   // If requested a move and arrived, start actuation once
227   if (moveRequested && !actuating &&
228       stepperX.distanceToGo() == 0 && stepperY.distanceToGo() == 0) {
229     startActuation();
230   }
231
232   // Run servo/magnet state machine
233   if (actuating) {
234     runActuation();
235   }
236
237   // keypad
238   char k = keypad.getKey();
239   if (!k) return;
240
241   Serial.print("Key: ");
242   Serial.println(k);
243
244   if (k == '*') {
245     clearInput();
246     return;
247   }
248
249   if (k >= '0' && k <= '9') {
250     if (idx < 2) {
251       buf[idx++] = k;
252       buf[idx] = '\0'; // keep it printable as a C-string
253       Serial.print("Input: ");
254       Serial.println(buf);
255     }
256     return;
257   }
258
259   if (k == '#') {
260     if (idx != 2) {
261       Serial.println("Need 2 digits (RC). Example: 12");
262       clearInput();
263       return;
264     }
265
266     int row = buf[0] - '0';
267     int col = buf[1] - '0';
268     clearInput();
269
270     if (row < 1 || row > 8 || col < 1 || col > 8) {
271       Serial.println("Row/Col must be 1..8");
272       return;
273     }
274
275     long x = (col - 1) * STEPS_PER_SQUARE_X;
276     long y = (row - 1) * STEPS_PER_SQUARE_Y;
277
278     stepperX.moveTo(x);
279     stepperY.moveTo(y);
280
281     moveRequested = true;
282
283     // Used for debugging and outputting to us what is registered input
284     Serial.print("Go to ");
285     Serial.print(row);
286     Serial.print(" ");
287     Serial.print(col);
288     Serial.print(" -> X=");
289     Serial.print(x);
290     Serial.print(" Y=");
291     Serial.println(y);
292   }
293 }
294

```